

Optimization Methods for LASSO

A Comparison of Frank-Wolfe, Pairwise Frank-Wolfe, and Projected
Gradient

Optimization for Data Science (2025/2026)

Authors

Christina Caporale, ID: 2141881
Natalya Lavrenchuk, ID: 2141882
Dumitru Movila, ID: 2144565

University of Padua
Department of Mathematics
Instructors: Francesco Rinaldi, Gannaro Auricchio

July 10, 2026

Contents

1	Introduction	2
2	Problem Formulation	2
2.1	Constrained LASSO	2
2.2	Geometry of the ℓ_1 -ball	3
2.3	Properties	3
3	Optimization Methods: Wolfe and Friends	4
3.1	Classical Frank-Wolfe (FW)	4
3.1.1	Algorithm Overview	4
3.1.2	Convergence Analysis	5
3.2	Pairwise Frank-Wolfe (PFW)	6
3.2.1	Algorithm Overview	6
3.2.2	Convergence Analysis	7
3.3	Projected Gradient (PG)	7
3.3.1	Algorithm Overview	7
3.3.2	Convergence Analysis	8
4	Results and discussion	9
4.1	Experimental Protocol	9
4.2	Dataset 1: Diabetes Indicators	10
4.3	Dataset 2: Communities and Crime	12
4.4	Dataset 3: Riboflavin Production	13
5	Conclusions	16
6	List of variables	17
7	LLM usage disclosure statement	17
	References	18

1 Introduction

Nearly 70 years after its introduction by Marguerite Frank and Philip Wolfe in 1956, the Frank-Wolfe algorithm remains a fundamental method for constrained convex optimization. Originally developed for quadratic programming over polytopes, the method has since been generalized to smooth convex optimization over convex sets and has demonstrated its utility in a wide variety of applications. With the rapid growth of large-scale constrained optimization problems in the modern era, interest has renewed in this older algorithm. Here, we demonstrate its relevance and adaptability across three experimental problems.

The Frank-Wolfe algorithm is an iterative, first-order optimization algorithm that possesses several beneficial characteristics, as compared to projected gradient methods:

1. **Projection-free optimization:** First, Frank-Wolfe is a projection-free algorithm. Rather than projecting an iterate back onto the feasible set after each update, it identifies the feasible point that minimizes the local linearized objective. For high-dimensional feasible sets, solving the Linear Minimization Oracle (LMO) can be less expensive than computing a projection, which requires a quadratic optimization problem, resulting in a lower per-iteration computational cost.
2. **Sparse iterates:** Second, the algorithm produces sparse iterates, making it useful for high-dimensional, real-world datasets. Since each iteration adds at most one vertex to the convex combination, the number of non-zero entries grows at most linearly with iterations, resulting in a model that is interpretable, fast, and memory efficient.
3. **A "free" optimality gap:** The duality gap measures first-order optimality and is already computed within the LMO. This no-cost calculation provides a convenient stopping criteria as it provides a certificate of optimality.
4. **Affine invariance:** The algorithm's behavior and convergence are unaffected by affine transformations of the coordinate system, meaning its performance does not depend on the coordinate representation.

These algorithmic properties, arising directly from the algorithms' design choices, are particularly valuable in modern real-world applications, which often involve high-dimensional, noisy datasets requiring robust and efficient optimization methods. However, the classical Frank-Wolfe approach can exhibit sublinear convergence when the solution lies on the boundary of the feasible set, motivating the study of improved variants. In this project, the classical Frank-Wolfe (FW), Pairwise Frank-Wolfe (PFW), and Projected Gradient (PG) algorithms are implemented and evaluated on a constrained LASSO problem applied to three real-world datasets. Their convergence behavior, computational performance, and solution quality are compared.

2 Problem Formulation

2.1 Constrained LASSO

Regression models are widely used throughout engineering and scientific fields to model relationships between variables and responses. The Least Absolute Shrinkage and Selection Operator (LASSO), shown in Equation (1), extends ordinary least squares regression by introducing an ℓ_1 constraint on regression coefficients. This constraint encourages sparse solutions, performing variable selection and shrinking coefficient magnitudes. The resulting models

are more interpretable and often have improved predictive performance in high-dimensional settings. Similarly, these methods are well suited to approximation problems, such as signal approximation and image compression, where the ultimate goal is sparse representation.

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) := \|Ax - b\|_2^2 \\ \text{s.t.} \quad & \|x\|_1 \leq \tau \end{aligned} \tag{1}$$

2.2 Geometry of the ℓ_1 -ball

LASSO is formulated as the minimization of the least-squares objective over an ℓ_1 norm ball, as defined below in Equation (2), representing the sum of absolute values of the vector components.

$$\mathcal{C} = \{x \in \mathbb{R}^n : \|x\|_1 \leq \tau\} = \text{conv}(\{\pm \tau e_i, i = 1, \dots, n\}) \tag{2}$$

The regularization parameter τ controls the size of the feasible region and, consequently, the amount of shrinkage forced upon the model. Smaller values of τ force stronger regularization by restricting the feasible region, while larger values permit a wider range of coefficient vectors. In two dimensions, the feasible region is diamond-shaped, while in higher dimensions it forms a bounded, convex polytope. Unlike the smooth boundary of an Euclidean ball, the ℓ_1 ball has sharp corners located on the coordinate axes. As a result, optimal solutions frequently occur at these corners, causing one or more coefficients to become exactly zero, leading to sparse solutions.

Geometrically, the ℓ_1 ball can be understood as the convex hull of the signed coordinate basis vectors scaled by τ . Therefore, the ℓ_1 ball has $2n$ atoms, corresponding to the positive and negative coordinate basis vectors. These signed basis vectors are the extreme points of the feasible region and every feasible point can therefore be expressed as a convex combination of these points. Frank-Wolfe is particularly well suited for optimization over convex hulls because each iteration only requires solving a linear minimization problem over the feasible region, avoiding expensive Euclidean projections. Since the ℓ_1 ball is the convex hull of its signed basis vectors, its geometry is especially favorable for Frank-Wolfe.

2.3 Properties

For this LASSO formulation, the gradient is shown below in Equation (3). The gradient indicates the direction of steepest increase of the objective function and defines the linear objective solved by the Frank-Wolfe LMO, which returns the next search direction.

$$\nabla f(x) = 2A^\top(Ax - b) \tag{3}$$

Frank-Wolfe convergence theory assumes that the objective function is smooth. Smoothness is evaluated through Lipschitz continuity of the gradient, meaning the gradient varies at a bounded rate and is calculated via the Hessian in Equation (4). This constant is later used to determine an appropriate step size for algorithms that account for the curvature of the objective function.

$$L = \|\nabla^2 f(x)\|_2 = 2\sigma_{\max}(A)^2, \quad \nabla^2 f(x) = 2A^\top A \tag{4}$$

The constrained LASSO problem satisfies the assumptions required by the Frank-Wolfe algorithm. The least-squares objective is convex and continuously differentiable with a Lipschitz continuous gradient, while the ℓ_1 ball is a compact, convex feasible region. Furthermore, because

the feasible region is the convex hull of the signed basis vectors, the LMO has a simple closed-form solution, selecting the signed basis vector of the largest magnitude component of the gradient. These properties make constrained LASSO a natural application for projection-free Frank-Wolfe optimization.

3 Optimization Methods: Wolfe and Friends

All three optimization algorithms considered in this work optimize the same objective and differ only in how each iteration is computed. We solve a general constrained convex optimization problems of the form:

$$\min_{x \in \mathcal{C}} f(x). \quad (5)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function and $\mathcal{C} \subseteq \mathbb{R}^n$ is a convex feasible set. At an optimal solution, no feasible direction exists that decreases the objective:

$$D(x^*) \cap F(x^*) = \emptyset. \quad (6)$$

We compare three first order methods: Classical Frank-Wolfe (FW), a variant called Pairwise Frank-Wolfe (PFW), and a Projected Gradient (PG) method.

3.1 Classical Frank-Wolfe (FW)

3.1.1 Algorithm Overview

Frank-Wolfe minimizes a linear approximation of the objective function over the feasible set. This linear minimization subproblem is solved using a Linear Minimization Oracle (LMO). It is generally defined in Equation (7) and the first-order Taylor approximation of the objective is written out in Equation (8).

$$s_k = \arg \min_{s \in \mathcal{C}} \langle s, \nabla f(x_k) \rangle \quad (7)$$

$$f(x_k) + \langle s - x_k, \nabla f(x_k) \rangle \quad (8)$$

Frank-Wolfe is a particularly efficient optimization strategy over atomic convex hulls because linear functions reach their optimum at extreme points. For the ℓ_1 ball, the atoms are the signed coordinate basis vectors, so the LMO simply selects the signed basis vector corresponding to the largest absolute component of the gradient. The closed form solution for the ℓ_1 -ball is shown in Equation (9). In this case, the LMO has computational complexity $O(n)$, since it only requires scanning the gradient to identify the largest component.

$$s_k = \tau \operatorname{sign}(-\nabla_{i_k} f(x_k)) e_{i_k}, \quad i_k = \arg \max_i |\nabla_i f(x_k)|. \quad (9)$$

The algorithm generates a sequence of feasible iterates, as demonstrated in Algorithm 1. At iteration k , the LMO identifies the feasible point minimizing the linearized objective. The point returned by the LMO is denoted by s_k and represents the Frank-Wolfe search point. The descent direction is defined in Equation (10) and the update step in Equation (11). The next iterate is obtained by taking a convex combination of the current iterate and this search point and since $\alpha_k \in [0, 1]$, every iterate is also feasible. Since each iteration introduces at most one new atom into the active set, the iterate after k iterations can be expressed as a convex combination of at most $k + 1$ atoms.

$$d_k = d_k^{FW} := s_k - x_k \quad (10)$$

$$x_{k+1} = x_k + \alpha_k(s_k - x_k) = \alpha_k s_k + (1 - \alpha_k)x_k \quad (11)$$

For step size selection, a simple choice is diminishing step size $2/(k+2)$. However here, because the LASSO objective is quadratic, the one-dimensional optimization along the Frank-Wolfe direction has a closed-form solution. Therefore, we employ exact line search at every iteration, selecting the step size that minimizes the objective along the Frank-Wolfe direction. The Frank-Wolfe gap that provides a certificate of first-order optimality is used as the stopping criterion.

Algorithm 1 Classical Frank-Wolfe Method for the ℓ_1 -Ball

```

1: Initialize:  $x_1 = \pm \tau e_i$ 
2: for  $k = 1, 2, \dots$  do
3:   LMO: Compute  $i_k = \arg \max_i |\nabla_i f(x_k)|$ ,  $s_k = \tau \operatorname{sign}(-\nabla_{i_k} f(x_k))e_{i_k}$ 
4:   FW gap:  $G(x_k) = -\nabla f(x_k)^\top (s_k - x_k)$ 
5:   if  $G(x_k) \leq \varepsilon$  then
6:     return  $x_k$ 
7:   end if
8:   Line search:  $\alpha_k = \arg \min_{\alpha \in [0,1]} f(x_k + \alpha(s_k - x_k))$ 
9:   Update:  $x_{k+1} = x_k + \alpha_k(s_k - x_k)$ 
10: end for

```

3.1.2 Convergence Analysis

Any convex optimization problem with an efficient LMO is a good candidate for Frank-Wolfe. Algorithm convergence efficiency depends on the problem formulation and the geometry of the feasible set. Convergence is defined with an optimality certificate called the Frank-Wolfe gap, as shown in Equation (12). This quantity measures the largest first-order improvement predicted by the linear approximation of the objective over the feasible region. Due to the convexity of the objective, this provides an upper bound on the unknown primal error. Since the FW vertex is already computed by the LMO, this gap calculation is obtained at no additional computational cost. It is always nonnegative, and equals zero only at a first-order optimal point. In practice, the FW gap is commonly used as the stopping criterion once the estimated optimality gap falls below a specified tolerance.

$$G(x) = \max_{s \in \mathcal{C}} -\nabla f(x)^\top (s - x) \quad (12)$$

This duality gap provides a computable bound on the sub-optimality, relating the easily computed FW gap directly to the unknown primal optimality gap and ultimately the true optimization error:

$$G(x) \geq f(x) - f(x^*). \quad (13)$$

The standard convergence rate for classical Frank-Wolfe is sublinear, decreasing proportionally to $1/k$:

$$f(x_k) - f^* \leq \frac{2C_f}{k+2} = O(1/k). \quad (14)$$

This is not only a worst-case result, but commonly occurs when the optimal solution lies on a flat face of the feasible set rather than at an extreme vertex. Since each iteration of FW

moves only toward a single vertex, the iterates begin to zigzag along the boundary, causing sluggish convergence. This behavior motivated the development of several Frank-Wolfe variants, discussed in the next section. In certain situations, such as when the optimum lies in the interior of the feasible set and the objective is strongly convex, classical Frank-Wolfe can achieve linear convergence.

The convergence rate is influenced by the curvature constant, which measures the deviation of the objective function from its first-order linear approximation over the feasible domain. It is closely related to both the smoothness of the objective function through the Lipschitz constant of the gradient and the size of the feasible set through its diameter as shown in Equation (15). The smoother objective functions and smaller feasible regions lead to smaller curvature constants and tighter convergence bounds.

$$C_f \leq L M^2, \quad M = \text{diam}(\mathcal{C}). \quad (15)$$

3.2 Pairwise Frank-Wolfe (PFW)

3.2.1 Algorithm Overview

Variants were later introduced to overcome a well-known limitation of the classical Frank-Wolfe algorithm: zigzagging behavior near the boundary of the feasible set. Wolfe first introduced the Away-step Frank-Wolfe (AFW) algorithm, allowing the method to move away from unfavorable vertices in the current active set and improving convergence near boundary solutions. The away vertex is chosen as the active atom with the largest inner product with the gradient, representing the least favorable active vertex. The active set refers to the collection of vertices whose convex combination represents the current iterate. Here, however, we focus another variant, the Pairwise Frank-Wolfe (PFW) algorithm.

Rather than alternating between a forward Frank-Wolfe step and an away step, PFW combines both into a single update. And unlike AFW, which redistributes weight across all active atoms during an away step, PFW transfers weight directly between two atoms. At each iteration, the algorithm queries the LMO to identify the best new search vertex while simultaneously selecting the least favorable vertex from the active set. The search direction then transfers weight from the away vertex to the new Frank-Wolfe vertex, swapping weight between the two atoms while maintaining feasibility. If the entire weight of the away atom is transferred, that atom is removed from the active set. This more aggressively prunes the active set, often producing sparser iterates and improving convergence in practice. This step is shown below in Equation (16). The maximum feasible step size is limited by the weight of the away atom, ensuring that no coefficient becomes negative and that the iterate remains a feasible convex combination of the active atoms.

$$d_k = s_k - v_k, \quad v_k \in \arg \max_{v \in A_k} \nabla f(x_k)^\top v \quad (16)$$

The algorithm is laid out below:

Algorithm 2 Pairwise Frank-Wolfe Method for the ℓ_1 -Ball

```
1: Initialize:  $x_1 = \pm \tau e_i$ ,  $A_1 = \{x_1\}$ ,  $\lambda_{x_1} = 1$ 
2: for  $k = 1, 2, \dots$  do
3:   LMO: Compute  $s_k$  using Eq. (9)
4:   FW gap: Compute  $G(x_k)$  using Eq. (12)
5:   if  $G(x_k) \leq \varepsilon$  then
6:     return  $x_k$ 
7:   end if
8:   Away step: Select  $v_k$  using Eq. (16)
9:   Step size: Set  $\alpha_{\max} = \lambda_{v_k}$  and compute  $\alpha_k$  by exact line search over  $[0, \alpha_{\max}]$ 
10:  Update:  $x_{k+1} = x_k + \alpha_k(s_k - v_k)$ 
11:  Active set: Update  $\lambda_{s_k}$  and  $\lambda_{v_k}$ ; remove  $v_k$  if  $\lambda_{v_k} = 0$  and add  $s_k$  if necessary
12: end for
```

3.2.2 Convergence Analysis

Improving upon classical FW, PFW can achieve a global linear convergence rate for strongly convex objectives over polyhedral feasible sets, by allowing weight to be transferred directly between active atoms, thereby alleviating the zig-zagging behavior observed near boundary solutions. The convergence behavior is governed by Equation (17), which depends on both the geometry of the feasible set and the properties of the objective function. Here, μ denotes the strong convexity constant, L the Lipschitz constant of the gradient, δ the pyramidal width, and M the diameter of the feasible set. The pyramidal width is a geometric property of the polytope that measures how favorable the geometry of the feasible set is for pairwise search directions. Larger values correspond to faster theoretical convergence.

$$h_k := f(x_k) - f(x^*), \quad h_{k+1} \leq (1 - \rho)h_k, \quad \rho = \frac{\mu}{4L} \left(\frac{\delta}{M} \right)^2. \quad (17)$$

Global linear convergence requires two assumptions: a polyhedral feasible set, which is satisfied here since the ℓ_1 -ball is a polytope, and sufficient curvature in the objective. For the least-squares objective considered here, sufficient curvature is provided by μ -strong convexity, which holds when the design matrix A has full column rank. When this condition is not met, f is no longer strongly convex, so the standard linear convergence guarantee does not apply. However, our objective has the composite form $f(x) = g(Ax)$, where the outer function g is strongly convex regardless of the rank of A . Rather than requiring strong convexity of f , the composite result requires only that g be strongly convex. Exploiting the affine invariance of Pairwise Frank-Wolfe, Lacoste-Julien and Jaggi [6] extend the convergence analysis to this composite setting, showing that PFW retains global linear convergence even when A is rank deficient. In Equation (17), the strong convexity constant μ is replaced by a generalized constant involving the Hoffman constant of A , allowing the convergence guarantee to extend to rank-deficient problems. Consequently, PFW retains a linear convergence guarantee for all datasets considered in this work.

3.3 Projected Gradient (PG)

3.3.1 Algorithm Overview

Here, we additionally examine the Projected Gradient Method as a point of comparison to the Frank-Wolfe approaches. Unlike Frank-Wolfe, which maintains feasibility at every iteration, Projected Gradient approaches first take an unconstrained gradient descent step, which may

leave the feasible set. The resulting point is then projected back onto the feasible set by finding the closest feasible point via minimizing the Euclidean distance. The projection operation is defined below in Equation (18). It is continuous and non-expansive, ensuring that projecting onto the feasible set does not increase distances between iterates.

$$\Pi_{\mathcal{C}}(\bar{x}) = \arg \min_{z \in \mathcal{C}} \frac{1}{2} \|z - \bar{x}\|^2 \quad (18)$$

The projected gradient step is then:

$$\hat{x}_k = \Pi_{\mathcal{C}} \left(x_k - \frac{1}{L} \nabla f(x_k) \right), \quad g_{\mathcal{C}}(x_k) = x_k - \hat{x}_k. \quad (19)$$

After taking a gradient step, the projected point becomes the next iterate:

$$x_{k+1} = \hat{x}_k = \Pi_{\mathcal{C}} \left(x_k - \frac{1}{L} \nabla f(x_k) \right). \quad (20)$$

The algorithm is written out below in Algorithm 3. Since the projection already incorporates the step size $1/L$, the projected point becomes the next iterate. The stopping criterion is based on the gradient mapping, whose norm equals zero if and only if the first-order optimality conditions are satisfied.

Algorithm 3 Projected Gradient Method

```

1: Initialize: Choose a feasible point  $x_1 \in \mathcal{C}$ 
2: for  $k = 1, 2, \dots$  do
3:   Projection: Compute  $\hat{x}_k = \Pi_{\mathcal{C}}(x_k - \frac{1}{L} \nabla f(x_k))$ 
4:   Gradient mapping: Compute  $g_{\mathcal{C}}(x_k) = x_k - \hat{x}_k$ 
5:   if  $\|g_{\mathcal{C}}(x_k)\| \leq \varepsilon$  then
6:     return  $x_k$ 
7:   end if
8:   Update: Set  $x_{k+1} = \hat{x}_k$ 
9: end for

```

3.3.2 Convergence Analysis

The PG method converges when the current point is unchanged by taking a gradient step followed by projection. Equivalently, the gradient at the solution is orthogonal to the boundary of the feasible set, so the projection returns the same point. This optimality condition is measured using gradient mapping as shown in Equation (21), which quantifies the difference between the current iterate and its projected gradient step. The gradient mapping serves a similar role to the Frank-Wolfe gap, providing a computable measure of optimality and a practical stopping criterion and also impacting next iteration calculation.

$$g_{\mathcal{C}}(x) = x - \Pi_{\mathcal{C}} \left(x - \frac{1}{L} \nabla f(x) \right). \quad (21)$$

For smooth convex objectives, the PG achieves the same sublinear convergence rate as unconstrained gradient descent, proportional to $O(1/k)$ and shown in Equation (22). For strongly convex objectives however, the convergence rate becomes linear. Strong convexity ensures the objective has sufficient curvature around the optimum, guaranteeing that every iteration makes a proportional reduction in the optimization error and leading to linear convergence. Unlike Frank-Wolfe, whose iterations require only a LMO, each Projected Gradient iteration requires solving a quadratic Euclidean projection problem. Consequently, the computational

cost depends heavily on the complexity of the projection operator. Projection onto the ℓ_1 -ball can actually be quite efficient with specialized sorting algorithms, as implemented in our code. This makes our PG implementation very competitive with FW approaches. However, projections onto more complicated feasible sets can become very computationally expensive.

$$f(x_k) - f(x^*) \leq \frac{L\|x_1 - x^*\|_2^2}{2k}. \quad (22)$$

4 Results and discussion

4.1 Experimental Protocol

To ensure a fair comparison, all optimization algorithms were initialized from the same vertex of the ℓ_1 -ball. A convergence tolerance of $\epsilon = 10^{-4}$ and a maximum of 3000 iterations were used for all experiments.

In Table 1, we summarize some key features of each chosen dataset. The datasets span increasing problem dimensionality, enabling comparison of the optimization methods across low, medium, and high dimensional settings. The diabetes and communities datasets satisfy the regime *Samples* $\gg$ *Features*, whereas the riboflavin dataset satisfies the opposite, *Features* $\gg$ *Samples*, which will be relevant to the observed convergence behavior.

Table 1: Summary of the datasets used in the experiments.

Dataset	Samples	Features	Rank	$\ x_{\text{OLS}}\ _1$	τ	$\kappa(A)$	Characteristics
Diabetes	353	10	Full	2.4	1.5	20.7	Low-dimensional, Well-conditioned
Communities	1595	101	Full	11.0	2.5	230.8	Medium-dimensional, Ill-conditioned
Riboflavin	56	4088	Deficient	7.5	1.0	∞	High-dimensional, Rank-deficient

In standard predictive modeling, the selection of the constraint radius τ directly dictates model sparsity and is tuned to balance the bias-variance tradeoff. However, because the primary objective of this study is to compare optimization dynamics rather than to train a predictive model, our priority was to ensure the ℓ_1 constraint remained active. If τ were excessively large, the constraint would become inactive, reducing the problem to unconstrained least squares and masking the algorithmic differences between FW, PFW, and PG. Therefore, rather than cross-validating τ for optimal generalization, we deliberately selected dataset-specific values designed to place the constrained optimum x^* directly on the boundary of the feasible set.

For the full-rank diabetes and communities datasets, where $n > p$, the unconstrained least-squares solution is unique. To ensure that the ℓ_1 constraint was active, we chose τ smaller than the ℓ_1 norm of this solution, $\|x_{\text{OLS}}\|_1$. For diabetes, $\|x_{\text{OLS}}\|_1 \approx 2.35$, so choosing $\tau = 1.5$ guarantees that the constraint is active. For communities, $\|x_{\text{OLS}}\|_1 \approx 10.98$, so we selected $\tau = 2.5$. For the riboflavin dataset, where $p \gg n$, the unconstrained least-squares solution is not unique, so this approach cannot be used. Instead, we selected $\tau = 1.0$ based on the τ -sweep in Figure 1. These values were chosen to ensure that all experiments were performed in the constrained LASSO setting rather than to optimize predictive performance.

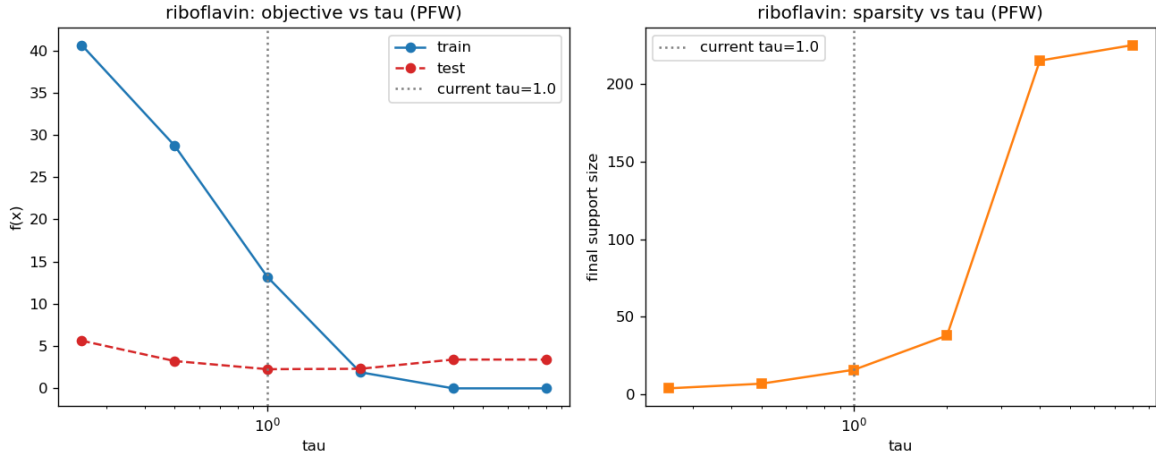


Figure 1: Effect of varying the regularization parameter τ on the riboflavin dataset.

To understand and compare the optimization difficulty of the three datasets, we also computed the condition number of the design matrix, $\kappa(A)$. Smaller condition numbers indicate a better-conditioned optimization problem, while larger values indicate an ill-conditioned problem. As shown in Table 1, the Diabetes dataset is comparatively well-conditioned versus the Communities dataset, which is substantially more ill-conditioned. The Riboflavin dataset is rank-deficient ($p \gg n$), giving $\kappa(A) = \infty$ because the smallest singular value is zero. These differences in problem conditioning help to explain the convergence behavior observed in the following experiments.

Additionally note that the algorithms use different optimality certificates. For FW and PFW we report the Frank-Wolfe gap, while for PG we report the norm of the projected-gradient mapping. In all cases, convergence is achieved when the corresponding certificate falls below the common tolerance $\epsilon = 10^{-4}$. Because the gradient mapping carries the $1/L$ step inside the projection, PG’s certificate at a given ϵ corresponds to a looser gradient-scale accuracy (of order $L\epsilon$) than the Frank-Wolfe gap, so iteration counts are not directly comparable across the two certificate families.

4.2 Dataset 1: Diabetes Indicators

Dataset description: This dataset contains measurements from 353 diabetes patients. Ten baseline variables including age, body mass index, blood pressure, and blood serum measurements are used to predict disease progression in one year [8]. Under an ℓ_1 -ball constraint with 10 predictor variables, the feasible set contains 20 atoms ($\pm e_i$), corresponding to the positive and negative coordinate directions. This is a low-dimensional, well-conditioned problem. A summary of the results from the experiment is listed below in Table 2.

Table 2: Performance comparison of the optimization algorithms on the diabetes dataset.

Algorithm	Objective	Iterations	Time (s)	Time/Iteration (μ s)	Support Size	Converged
FW	167.5517	3000	0.1642	54.7	10	No
PFW	167.4237	202	0.0265	131.2	10	Yes
PG	167.4241	234	0.0121	51.7	10	Yes

Runtime performance: The best performing algorithm, in terms of wall-clock runtime, was PG (0.01 s, 234 iterations), followed by PFW (0.03 s, 202 iterations), and lastly FW (0.16 s, 3000 iterations). FW and PG have very similar per-iteration times, and PFW costs about 2.5x more due to the additional calculations. PG achieved the shortest runtime by combining a relatively low iteration count with the lowest per-iteration cost.

Dataset results: Our final top 10 most impactful coefficients are shown in Figure 2. BMI and two blood serum measurements were found to be the most predictive variables in diabetes progression. All three models agree on these predictions, even for FW which did not entirely converge.

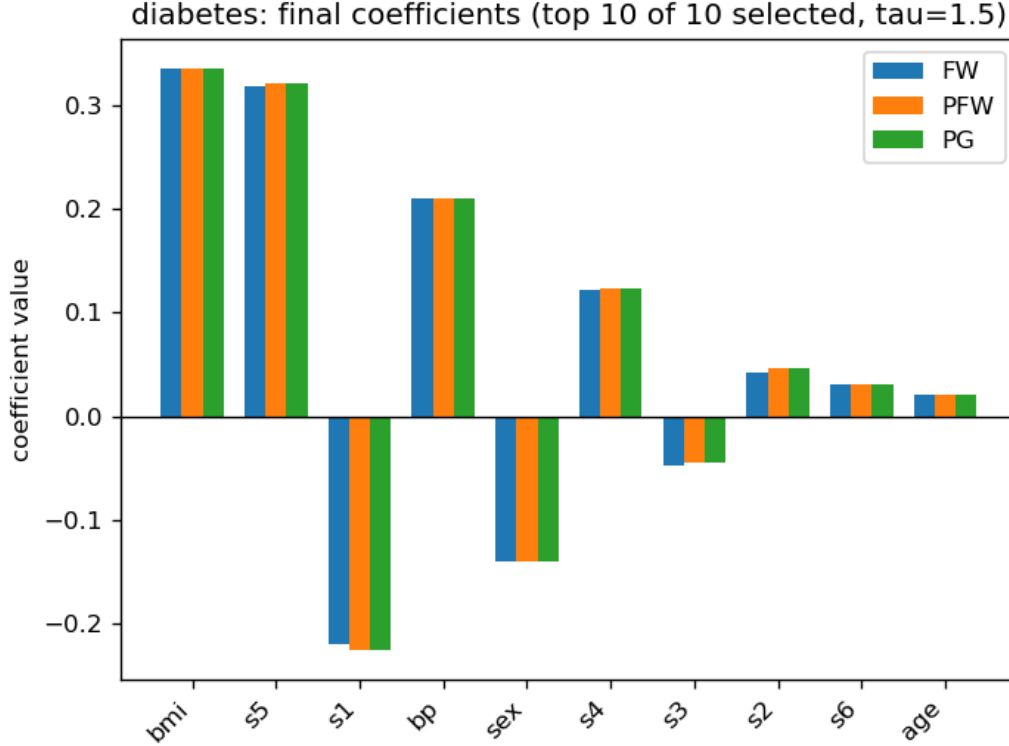


Figure 2: BMI, and S5 and S1 levels, are identified as the top three most predictive factors for the diabetes dataset.

Objective function behavior: As shown in Figure 3, all three algorithms rapidly reduce the objective function during the first few iterations before plateauing at nearly identical objective values. The similarity of the final objective values, together with the nearly identical selected features shown in Figure 2, suggests that all three methods identify essentially the same solution.

Convergence behavior: The stopping-criteria plot in Figure 3 reveals an important limitation of classical FW. The algorithm never converges within the 3000-iteration limit and exhibits the characteristic zig-zagging behavior that occurs when the solution lies on or near the boundary of the feasible region. Successive search directions become increasingly orthogonal to the gradient, resulting in inefficient oscillations and this leads to sublinear convergence. This example also illustrates the distinction between objective convergence and convergence

according to an optimality certificate. Although the objective value changes very little after the initial iterations, the Frank-Wolfe gap remains relatively large, preventing the stopping criterion from being satisfied and leading to a longer wall-clock time.

Pairwise Frank-Wolfe largely eliminates this zigzag behavior by allowing weight to be transferred between atoms, enabling convergence within only 202 iterations. However, it is not the fastest approach in this example, where PFW and PG both achieve strongly-convex linear rate. PG required more iterations yet took less time to converge, indicating that each iteration was actually cheaper than the PFW iteration in this example. FW and PFW are typically advantageous when projection onto the feasible set is computationally expensive, since each iteration requires only an LMO calculation. In this particular low-dimensional problem set, projection onto the ball is cheap, and projection ends up being faster than each PFW iteration.

Sparsity: All three methods converged to solutions containing ten nonzero coefficients, indicating that the chosen value of $\tau = 1.5$ was not sufficiently restrictive to induce a sparse solution. A smaller value of τ would shrink the feasible region, encouraging fewer nonzero coefficients and therefore a sparser model. The active-set graph in Figure 3 further highlights the difference between methods. FW only adds atoms to its active set, causing the support to always increase, whereas PFW can both add and remove atoms. The temporary increase to 11 active atoms followed by a reduction back to 10 demonstrates this behavior.

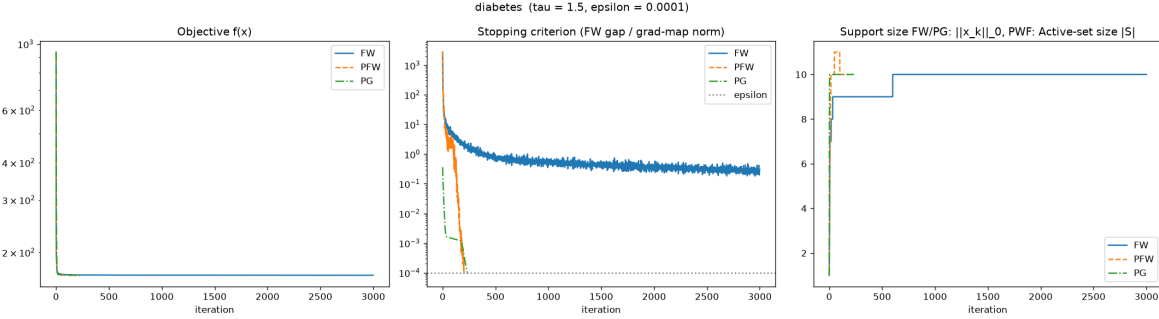


Figure 3: Performance of our three approaches on the diabetes dataset, as measured by a) objective function value, b) stopping criteria, and c) active-set size over iteration.

4.3 Dataset 2: Communities and Crime

Dataset description: The second dataset describes crime data from communities across the United States during the 1990’s. The predictor variables include socioeconomic, demographic, and law enforcement characteristics, such as median family income and the number of police officers per capita. The response variable is the per-capita violent crime rate, aggregating offenses such as murder, robbery, and assault. The original dataset contains 1,595 communities and 101 attributes, although after preprocessing only the numeric features were retained for the experiments [7]. This is a moderate-dimensional, ill-conditioned problem. A summary of the results from the experiment is listed below in Table 3.

Runtime performance: The best performing algorithm, in terms of wall-clock runtime, was PG (0.48 s, 995 iterations), followed by FW (1.41 s, 3000 iterations), and lastly PFW (1.91 s, 2959 iterations). Again, FW and PG have similar iteration times, and PFW takes about 1.5x longer. PG wins again due to fewer iterations required combined with shorter iterations.

Table 3: Performance comparison of the optimization algorithms on the communities dataset.

Algorithm	Objective	Iterations	Time (s)	Time/Iteration (μ s)	Support Size	Converged
FW	503.3919	3000	1.4093	469.8	62	No
PFW	501.3153	2959	1.9135	646.7	55	Yes
PG	501.6351	995	0.4781	480.5	64	Yes

Dataset results: The final top 20 most impactful coefficients are shown in Figure 4. The three methods selected the same top predictors, indicating that the optimization algorithm had little influence on the final variable ranking despite differences in convergence behavior.

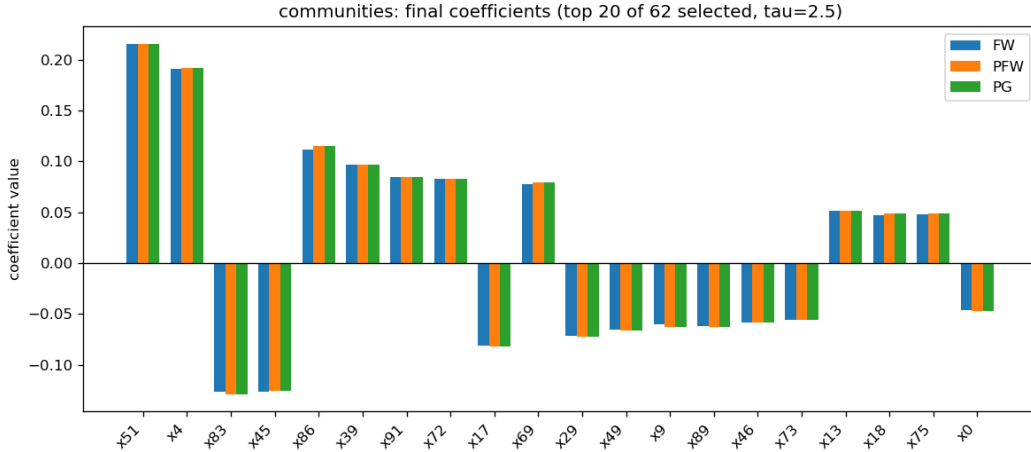


Figure 4: Top twenty most important features identified by each model for the communities dataset.

Objective function behavior: As shown in Figure 5, all three models showed fast decreases of the objective function and achieved similar objective values. FW failed to converge once again, remaining at a slightly higher objective function value as compared to PFW and PG.

Convergence behavior: Despite PFW’s theoretical linear convergence rate, the convergence constant depends on the conditioning of the problem. Because this dataset is relatively ill-conditioned, μ/L is small, reducing the practical advantage of its linear convergence rate and requiring nearly 3000 iterations. The contraction factor $\rho = (\mu/4L)(\delta/M)^2$ also carries a geometric penalty $(\delta/M)^2$ beyond the conditioning ratio μ/L ; the pyramidal width δ of the ℓ_1 -ball shrinks with dimension, which is why PFW needed far more iterations on the ~ 100 -dimensional communities problem than on the 10-dimensional diabetes one.

Sparsity: Pairwise Frank-Wolfe produced the sparsest solution, requiring only 55 active variables compared with 64 for Projected Gradient, while still achieving a comparable objective value as shown in Figure 5.

4.4 Dataset 3: Riboflavin Production

Dataset description: Our third experiment uses the riboflavin dataset, which contains gene expression measurements from *Bacillus subtilis* cultures, together with their riboflavin (vitamin

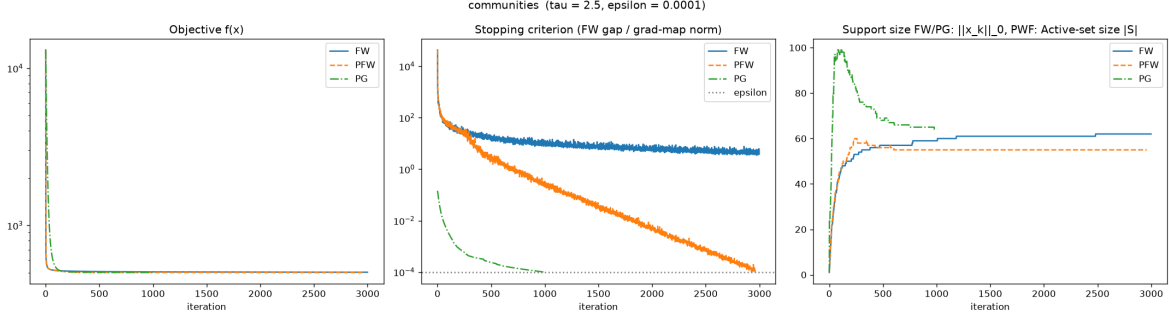


Figure 5: Performance of our three approaches on the communities dataset, as measured by a) objective function value, b) stopping criteria, and c) active-set size over iteration.

B2) production rates. The dataset consists of only 56 observations but 4088 gene-expression predictors [3]. Such high-dimensional genomic datasets are a natural application for LASSO regression, since sparse models improve interpretability by identifying a relatively small subset of influential genes. Because the number of predictors greatly exceeds the number of observations, the design matrix is rank deficient, making this a fundamentally different optimization problem from the previous two datasets. A summary of the results from the experiment is listed below in Table 4.

Table 4: Performance comparison of the optimization algorithms on the riboflavin production dataset.

Algorithm	Objective	Iterations	Time (s)	Time/Iteration (μ s)	Support Size	Converged
FW	13.1905	3000	1.7862	595.4	19	No
PFW	13.1788	330	0.2595	786.4	16	Yes
PG	15.4239	3000	2.9972	999.1	42	No

Runtime performance: The best performing algorithm, in terms of wall-clock runtime, was PFW (0.26 s, 330 iterations), followed by FW (1.79 s, 3000 iterations), and lastly PG (3.00 s, 3000 iterations). FW has the shortest per-iteration time. However, unlike the other experiments, PFW iterations are faster than those of PG, indicating that the projection step becomes more expensive in this high-dimensional setting. Together with its rapid convergence, this gives PFW the lowest overall runtime.

Dataset results: The final top 20 most impactful coefficients are shown in Figure 6. Here, we observe noticeable differences between the solutions identified by the Frank-Wolfe methods and PG, although only PFW actually converged.

Objective function behavior: As shown in Figure 7, PFW was the only algorithm to satisfy the stopping criterion within the 3000-iteration limit and achieved the lowest objective value. PG reduced the objective much more slowly, terminating at a noticeably higher value, while FW plateaued early because of its characteristic boundary zig-zagging. Unlike the lower-dimensional datasets, the projection step also became comparatively expensive, making PFW both the fastest-converging and the lowest-runtime method.

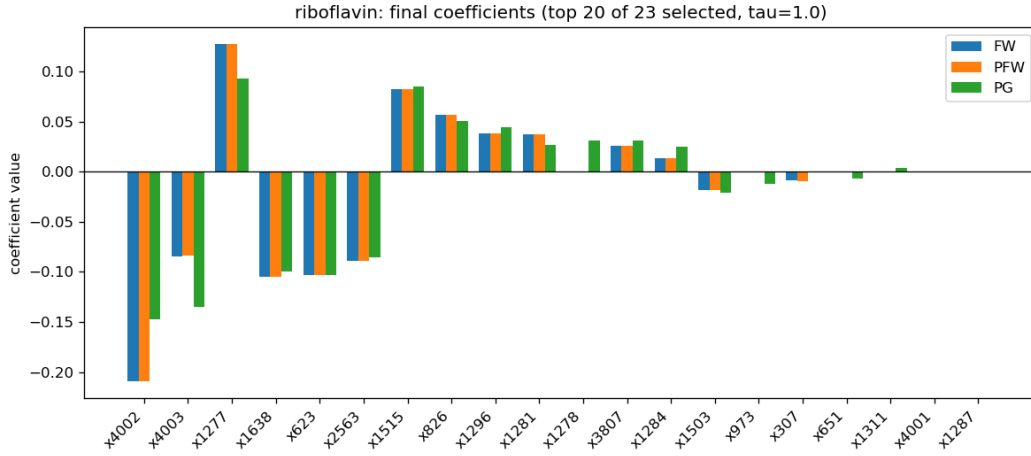


Figure 6: Top twenty most important genes identified by each model for the riboflavin dataset.

Convergence behavior: FW again failed to satisfy the stopping criterion, exhibiting the boundary zig-zagging discussed previously. PG also failed to converge within the iteration limit, whereas PFW rapidly reached the stopping criterion. This behavior matches the theoretical convergence guarantees discussed earlier in Section 3. Because the objective is not strongly convex for this rank-deficient problem, PG no longer has a linear convergence guarantee. In contrast, the composite convergence result for PFW still applies, allowing it to retain a global linear convergence guarantee despite the rank-deficient design.

Sparsity: Pairwise Frank-Wolfe achieved the lowest objective value and shortest runtime. It also produced the sparsest solution, requiring only 16 active atoms compared with 19 for FW and 42 nonzero coefficients for PG. These results demonstrate the additional sparsity advantages of PFW on this high-dimensional sparse regression problem.

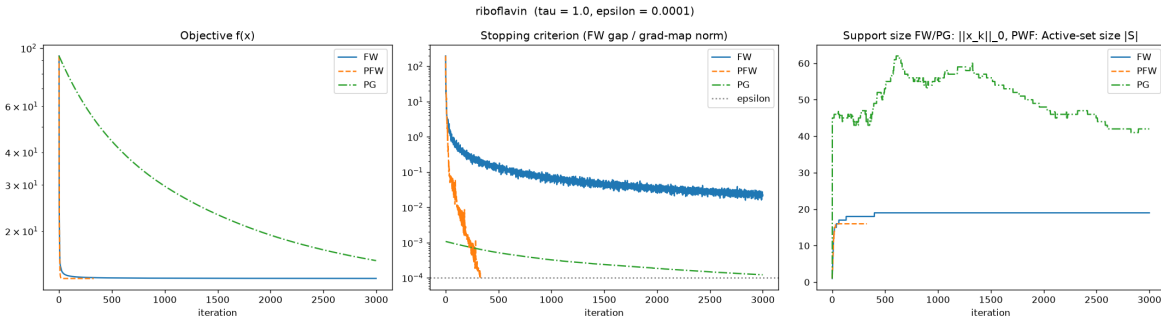


Figure 7: Performance of our three approaches on the riboflavin production dataset, as measured by a) objective function value, b) stopping criteria, and c) active-set size over iteration.

This dataset marks a shift in the best-performing algorithm from Projected Gradient to Pairwise Frank-Wolfe. In this high-dimensional, rank-deficient setting, PFW benefits from projection-free updates, sparse iterates, and a convergence guarantee that remains valid despite the loss of strong convexity, allowing it to outperform the other methods.

5 Conclusions

This work compared the Classical Frank-Wolfe (FW), Pairwise Frank-Wolfe (PFW), and Projected Gradient (PG) methods for solving constrained LASSO problems over the ℓ_1 -ball across three datasets spanning well-conditioned, ill-conditioned, and rank-deficient optimization regimes.

No Free Lunch! No single algorithm performed best across all three datasets. Projected Gradient performed best on the low dimensional diabetes and medium-dimensional communities datasets, where the objective was strongly convex and Euclidean projection onto the feasible set remained inexpensive. In contrast, Pairwise Frank-Wolfe achieved the strongest performance on the high-dimensional riboflavin dataset, where its projection-free iterations and sparse active-set representation became advantageous.

Theory matched experimental behavior. Classical Frank-Wolfe consistently exhibited its characteristic slow boundary convergence, reaching the maximum iteration limit on all three datasets despite producing competitive objective values. Pairwise Frank-Wolfe largely eliminated this zig-zagging behavior, converging on every dataset. Projected Gradient displayed the expected rapid convergence on the strongly convex Diabetes and Communities problems. However, on the rank-deficient Riboflavin dataset ($p \gg n$), PG lost its linear convergence guarantee and failed to converge within the iteration limit. In contrast, PFW retained a linear convergence guarantee through the composite objective formulation, allowing it to converge despite the loss of strong convexity. Finally, the observed convergence behavior reflected the conditioning of each problem, with the ill-conditioned Communities dataset requiring substantially more iterations than the comparatively well-conditioned Diabetes dataset despite both satisfying the assumptions for linear convergence.

Pairwise Frank-Wolfe provided the most robust overall performance. Although Pairwise Frank-Wolfe incurs additional work per iteration to maintain its active set, it consistently converged and produced sparse solutions across all three datasets. Its ability to transfer weight directly between active atoms substantially reduced the slow boundary zig-zagging exhibited by Classical Frank-Wolfe, while its composite convergence guarantee allowed it to remain effective even on the rank-deficient Riboflavin dataset.

Frank-Wolfe methods become increasingly attractive for large-scale, high-dimensional optimization problems where sparse solutions are desired. Projection-based methods remain highly effective for low and moderate-dimensional problems, while projection-free Frank-Wolfe variants become increasingly attractive as dimensionality grows and sparse solutions are desired. Overall, these experimental results demonstrate that the success of an optimization algorithm depends on many factors, with problem characteristics like dimensionality, conditioning, and rank structure often just as important as theoretical convergence guarantees. Convergence guarantees are asymptotic worst-case results, whereas practical performance depends on implementation, sparsity, conditioning, and the chosen tolerance. Each situation requires its own unique, carefully considered algorithm tuned to the specific properties of the dataset and the intended application.

6 List of variables

Symbol	Description
α_k	Step size
$\alpha_{\max}$	Maximum step size
C	Feasible set
C_f	Curvature constant
d_k	Search direction
d_k^{FW}	Frank–Wolfe direction
δ	Pyramidal width
ϵ	Convergence tolerance
f	Objective function
f^*	Optimal objective value
g	Transformed objective function
$g_C(x)$	Gradient mapping
$G(x)$	Frank–Wolfe gap
h_k	Primal error
i_k	Selected coordinate index
k	Iteration index
L	Lipschitz constant
M	Diameter of the feasible set
μ	Strong convexity constant
n	Number of variables
p	Number of features
Π_C	Projection onto C
ρ	Linear convergence rate
s	Point in the feasible set
s_k	Frank–Wolfe search vertex
τ	ℓ_1 -ball radius
v_k	Away atom
x	Optimization variable
$\hat{x}_k$	Projected point
x_k	Current iterate
x^*	Optimal solution
x_{OLS}	Unconstrained least-squares solution
$\nabla f(x)$	Objective gradient
$\nabla^2 f(x)$	Objective Hessian

7 LLM usage disclosure statement

During the development of this project our group made use of LLM’s such as ChatGPT (GPT-4o, OpenAI) and Claude (Sonnet 4.6, Anthropic). These were consulted intermittently throughout June and July 2026.

AI assistance was used strictly for study support in the following ways:

- LaTeX formatting and transcription of equations that were verified against course notes and standard references. This was used as an aid for the report, notebook markdowns,

and code comments.

- Limited assistance with programming syntax, debugging, and clarification of implementation details. This did not involve generating complete solutions or algorithmic code, and all code was written, understood, and validated by the team.
- Editing and refinement of the text, while the entire final content of the report reflects the own understanding of the authors.
- Discussing the choice of the most suitable datasets to showcase algorithm performance across optimization methods. The final datasets were validated by the team independently.

No AI tool was used to automatically draft algorithmic code in this assignment. All modeling choices, algorithm design, and final code reflect the independent work and understanding of the team.

References

- [1] I. M. Bomze, F. Rinaldi, and D. Zeffiro, *Frank-Wolfe and Friends: A Journey into Projection-Free First-Order Optimization Methods*, 4OR, vol. 19, no. 3, pp. 313-345, 2021. doi: 10.1007/s10288-021-00493-y.
- [2] Combettes, C. W., and Pokutta, S. (2021). *Complexity of Linear Minimization and Projection on Some Sets*. arXiv preprint arXiv:2101.10020.
- [3] CRAN. *Riboflavin: Riboflavin Data Set*. R package hdi documentation. Available at: <https://search.r-project.org/CRAN/refmans/hdi/html/riboflavin.html>. Accessed July 7, 2026.
- [4] F. Emmert-Streib and M. Dehmer, *High-Dimensional LASSO-Based Computational Regression Models: Regularization, Shrinkage, and Selection*, Machine Learning and Knowledge Extraction, vol. 1, no. 1, pp. 359-383, 2019. doi: 10.3390/make1010021.
- [5] Jaggi, M. (2013). *Revisiting Frank-Wolfe: Projection-Free Sparse Convex Optimization*. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, PMLR 28, 427-435.
- [6] Lacoste-Julien, S., and Jaggi, M. (2015). *On the Global Linear Convergence of Frank-Wolfe Optimization Variants*. Advances in Neural Information Processing Systems (NeurIPS), 28, 496-504.
- [7] Redmond, M. *Communities and Crime* [Dataset]. UCI Machine Learning Repository, 2002. <https://doi.org/10.24432/C53W3X>.
- [8] Scikit-learn Developers. Diabetes dataset. https://scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset, accessed July 6, 2026.
- [9] J. Duchi, S. Shalev-Shwartz, Y. Singer, and T. Chandra, *Efficient Projections onto the ℓ_1 -Ball for Learning in High Dimensions*, Proceedings of the 25th International Conference on Machine Learning, pp. 272-279, 2008. doi: 10.1145/1390156.1390191.
- [10] L. Condat, *Fast Projection onto the Simplex and the ℓ_1 Ball*, Mathematical Programming, vol. 158, no. 1, pp. 575-585, 2016. doi: 10.1007/s10107-015-0946-6.